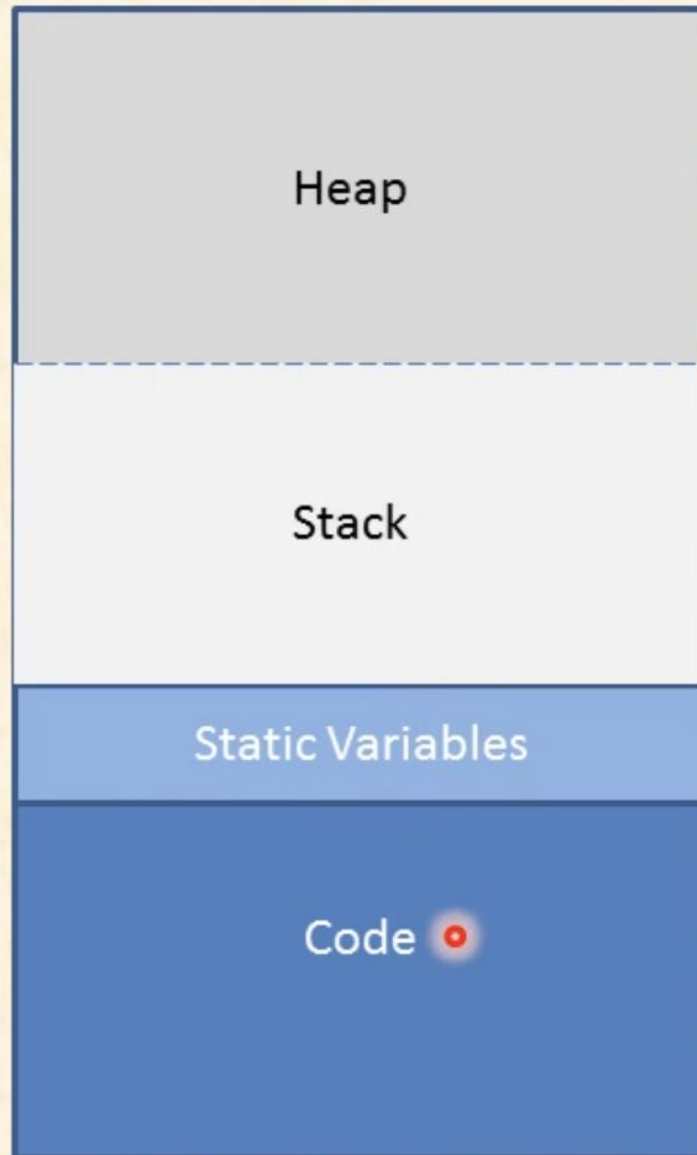


Memory

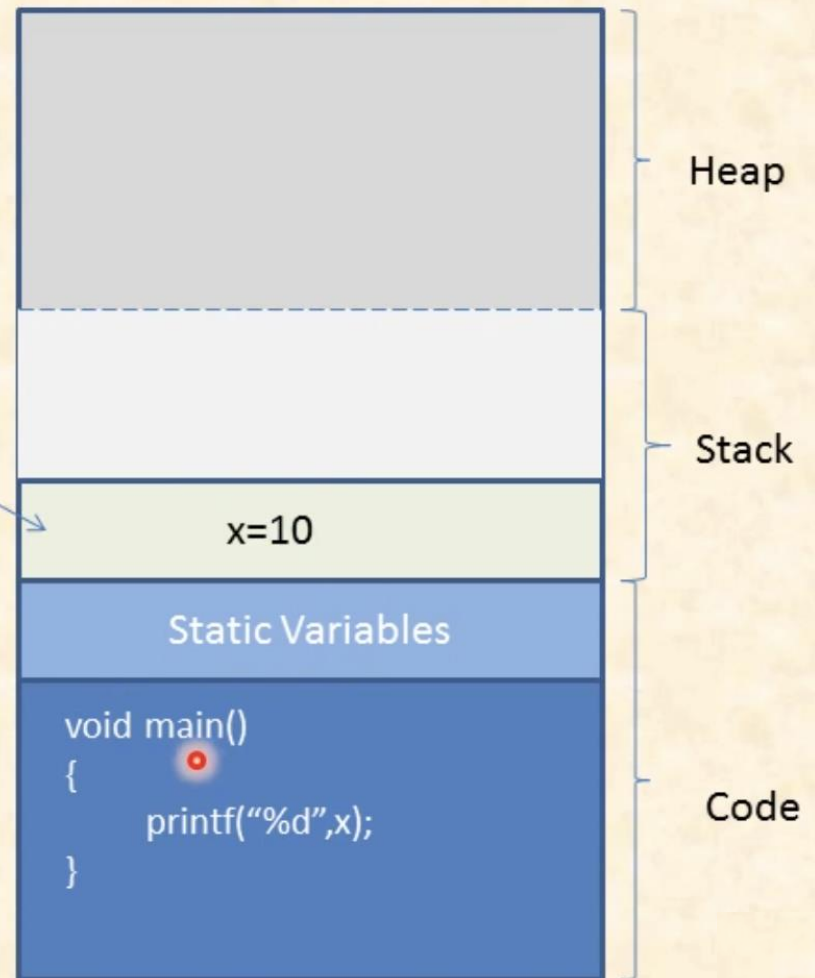


Loading of Program

```
void main()  
{  
    int x=10;  
    printf("%d",x);  
}
```

Activation
Record

Load



Recursion

```
int fun(int n)
```

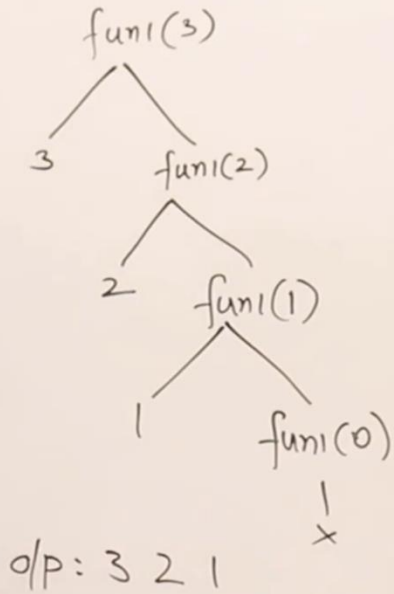
```
{  
    1. _____  
    2. _____  
    3. _____  
}
```

```
int main()
```

```
{  
    1. _____  
    2. _____ 10  
    3. fun(x) * 2;  
    4. _____  
    5. _____  
}
```

Recursion

Recursion



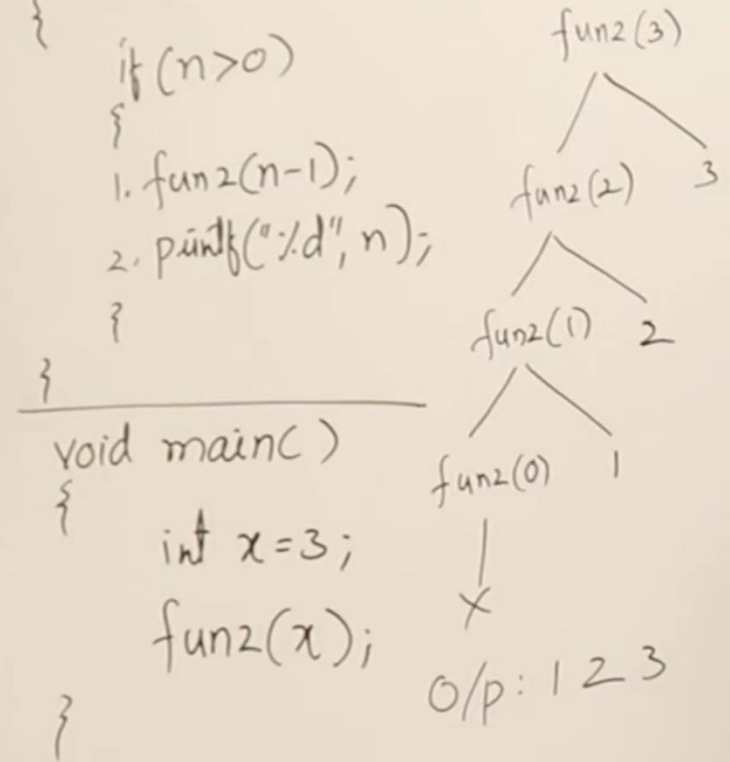
```

void fun1(int n)
{
    if (n > 0)
    {
        1. printf("%d", n);
        2. fun1(n-1);
    }
}

void main()
{
    int x=3;
    fun1(x);
}
  
```

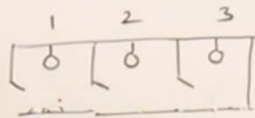
```

void fun2(int n)
{
    if (n > 0)
    {
        1. fun2(n-1);
        2. printf("%d", n);
    }
}
  
```

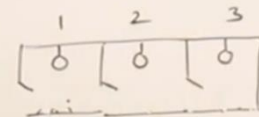


```

void main()
{
    int x=3;
    fun2(x);
}
  
```



1. Switch on Bulb
 → 2. goto Next Room
 o/p: 1 2 3



→ 1. goto Next Room
 2. Switch on Bulb
 o/p: 3 2 1

Recursion

```
void fun(int n)
{
```

```
    if (n > 0)
    {
```

Ascending 1. Calling

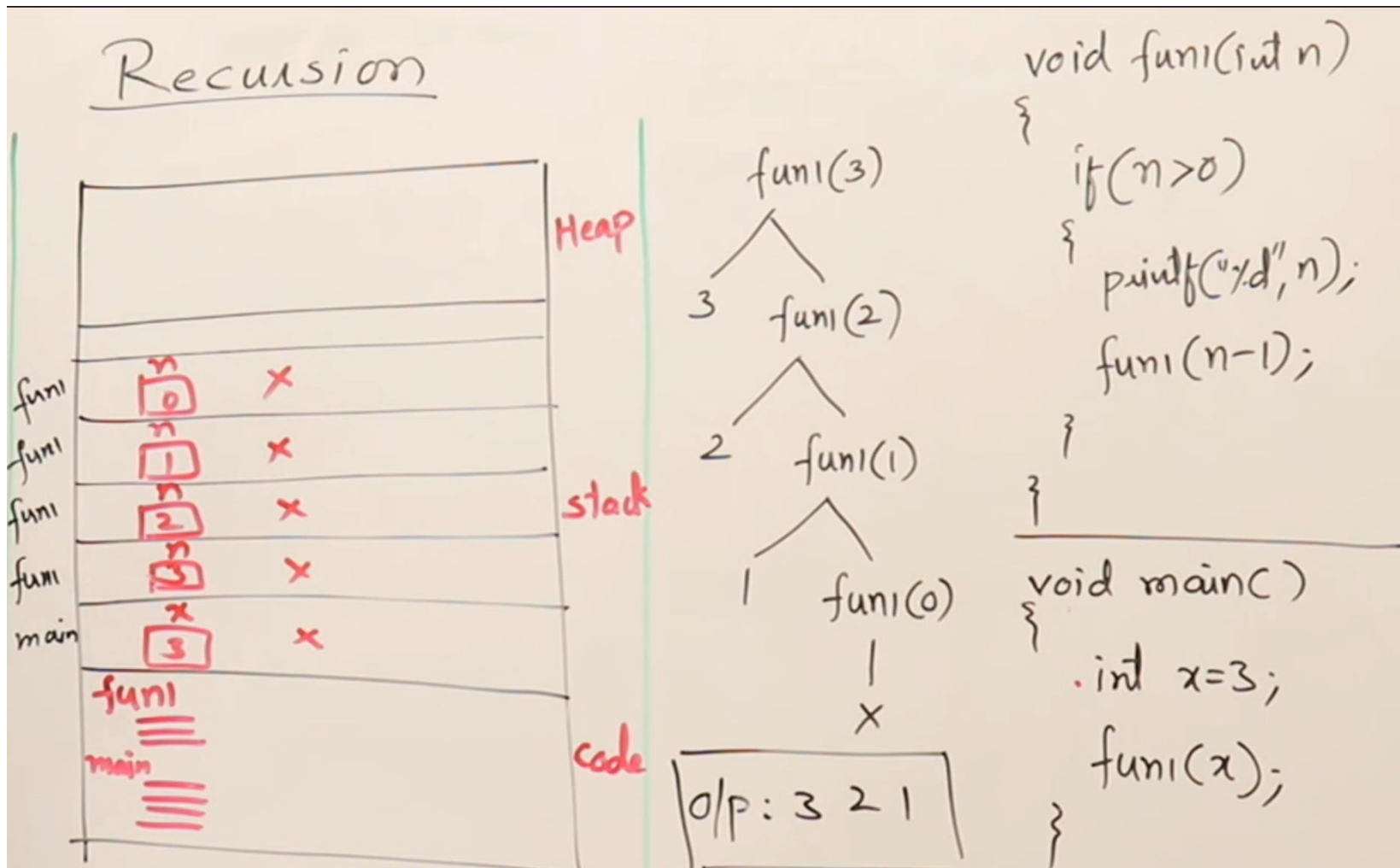
2. $\text{fun}(n-1)$ * 2

Descending 3. Returning ↑

```
    }
}
```

Recursion has both ascending and descending phases, while a for loop has only ascending phase.

Recursion



- ❑ Activation records are created in the stack for each function call.
- ❑ Once the function ends, the activation record is deleted from the stack.

Recursion

- Time complexity using recursion tree.

- Time complexity using recurrence relation.

```
void fun1(int n)
```

```
{  
    if(n > 0)  
    {  
        printf("%d", n);  
        fun1(n-1);  
    }  
}
```

```
{  
}
```

```
void main()  
{
```

```
    int x=3;
```

```
    fun1(x);
```

```
}
```

fun1(3)

3
└─ fun1(2)

2
└─ fun1(1)

1
└─ fun1(0)

└─ x

o/p: 3 2 1

1 →

1 →

1 →

3 units

n units

$O(n)$

$$T(n) = \begin{cases} 1 & n=0 \\ T(n-1) + 1 & n>0 \end{cases}$$

Solving Recurrences

- Recurrences are a major tool for **analysis** of algorithms.
 - Divide and Conquer algorithms are analyzable by recurrences.
- Also note, recurrence relations are used to **design** algorithms for combinatorial functions and can embody dynamic programming idea.

Methods for Solving Recurrences

- ▣ Using Substitution and Mathematical Induction
- ▣ Using Recursion-tree
- ▣ Using Master Theorem

Example: Integer Multiplication

- Let $X = \boxed{A}\boxed{B}$ and $Y = \boxed{C}\boxed{D}$ where A, B, C and D are $n/2$ bit integers
- **Simple Method:** $XY = (2^{n/2}A+B)(2^{n/2}C+D)$
- **Running Time Recurrence**

$$T(n) < 4T(n/2) + 100n$$

How do we solve it?

Divide-and-Conquer Multiplication: Warmup

To multiply two n -bit integers a and b :

- Multiply four $\frac{1}{2}n$ -bit integers, recursively.
- Add three pairs of $\frac{1}{2}n$ -bit integers and shift to obtain result.

$$\begin{aligned}x &= 2^{n/2} \cdot x_1 + x_0 \\y &= 2^{n/2} \cdot y_1 + y_0 \\xy &= (2^{n/2} \cdot x_1 + x_0)(2^{n/2} \cdot y_1 + y_0) = 2^n \cdot x_1 y_1 + 2^{n/2} \cdot (x_1 y_0 + x_0 y_1) + x_0 y_0\end{aligned}$$

Ex. $a = \underbrace{10001}_{x_1} \underbrace{1101}_{x_0} \quad b = \underbrace{11100001}_{y_1} \underbrace{}_{y_0}$

$$T(n) = \underbrace{4T(n/2)}_{\text{recursive calls}} + \underbrace{\Theta(n)}_{\text{add, shift}} \Rightarrow T(n) = \Theta(n^2)$$



assumes n is a power of 2

Substitution method

The most general method:

1. **Guess** the form of the solution.
2. **Verify** by induction.
3. **Solve** for constants.

Example: $T(n) = 4T(n/2) + 100n$

- [Assume that $T(1) = \Theta(1)$.]
- Guess $O(n^3)$. (Prove O and Ω separately.)
- Assume that $T(k) \leq ck^3$ for $k < n$.
- Prove $T(n) \leq cn^3$ by induction.

Example of substitution

$$\begin{aligned}T(n) &= 4T(n/2) + 100n \\&\leq 4c(n/2)^3 + 100n \\&= (c/2)n^3 + 100n \\&= cn^3 - ((c/2)n^3 - 100n) \quad \leftarrow \text{desired} - \text{residual} \\&\leq cn^3 \quad \leftarrow \text{desired}\end{aligned}$$

whenever $(c/2)n^3 - 100n \geq 0$, for
example, if $c \geq 200$ and $n \geq 1$. $\swarrow$ residual

Example (continued)

- We must also handle the initial conditions, that is, ground the induction with base cases.
- **Base:** $T(n) = \Theta(1)$ for all $n < n_0$, where n_0 is a suitable constant.
- For $1 \leq n < n_0$, we have “ $\Theta(1)$ ” $\leq cn^3$, if we pick c big enough.

This bound is not tight!

A tighter upper bound?

We shall prove that $T(n) = O(n^2)$.

Assume that $T(k) \leq ck^2$ for $k < n$:

$$\begin{aligned} T(n) &= 4T(n/2) + 100n \\ &\leq cn^2 + 100n \\ &\leq cn^2 \end{aligned}$$

for *no* choice of $c > 0$. Lose!

A tighter upper bound!

IDEA: Strengthen the inductive hypothesis.

- ***Subtract*** a low-order term.

Inductive hypothesis: $T(k) \leq c_1 k^2 - c_2 k$ for $k < n$.

$$\begin{aligned} T(n) &= 4T(n/2) + 100n \\ &\leq 4(c_1(n/2)^2 - c_2(n/2)) + 100n \\ &= c_1 n^2 - 2c_2 n + 100n \\ &= c_1 n^2 - c_2 n - (c_2 n - 100n) \\ &\leq c_1 n^2 - c_2 n \quad \text{if } c_2 > 100. \end{aligned}$$

Pick c_1 big enough to handle the initial conditions.

Recursion-tree method

- ❑ A recursion tree models the costs (time) of a recursive execution of an algorithm.
- ❑ The recursion tree method is good for generating guesses for the substitution method.
 - ❑ The recursion-tree method can be unreliable, just like any method that uses ellipses (...).
- ❑ The recursion-tree method promotes intuition, however.

Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



Example of recursion tree

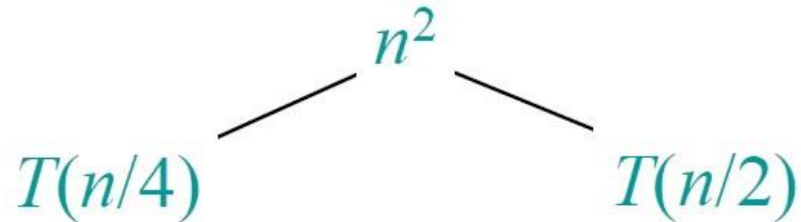
Solve $T(n) = T(n/4) + T(n/2) + n^2$:

$$T(n)$$



Example of recursion tree

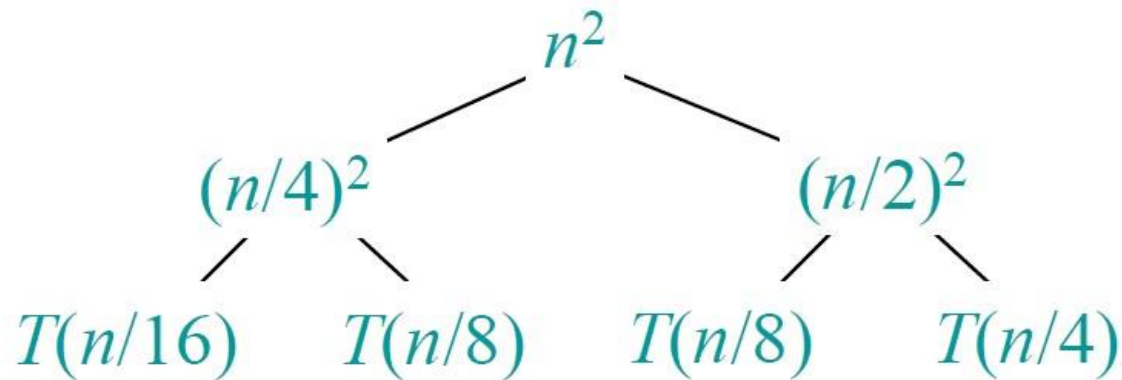
Solve $T(n) = T(n/4) + T(n/2) + n^2$:





Example of recursion tree

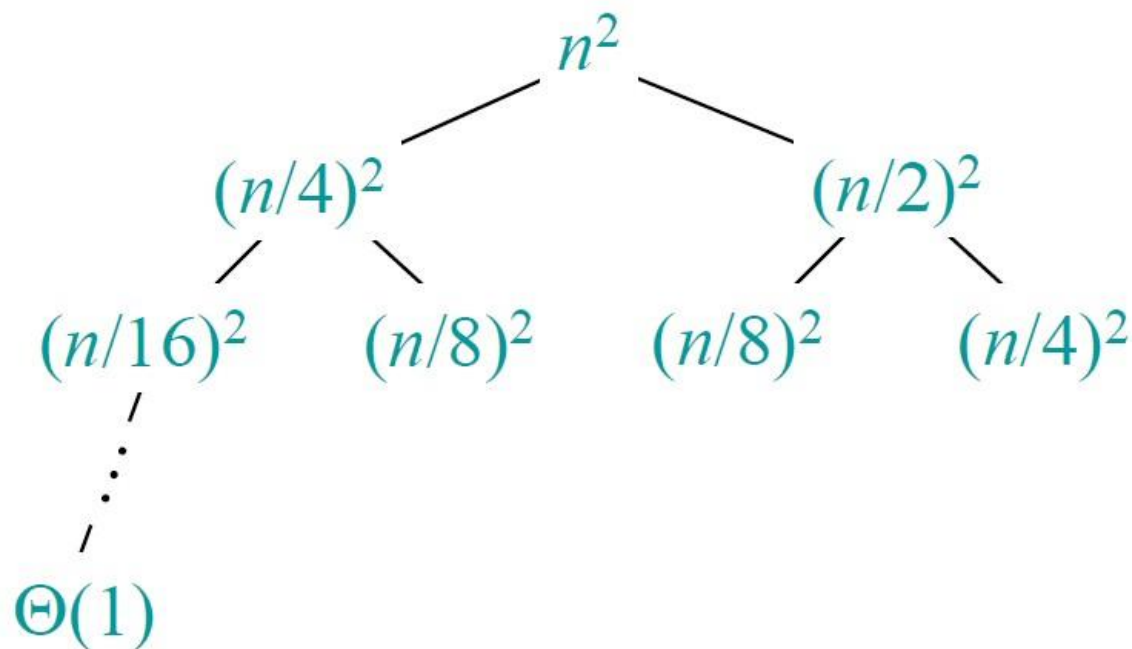
Solve $T(n) = T(n/4) + T(n/2) + n^2$:





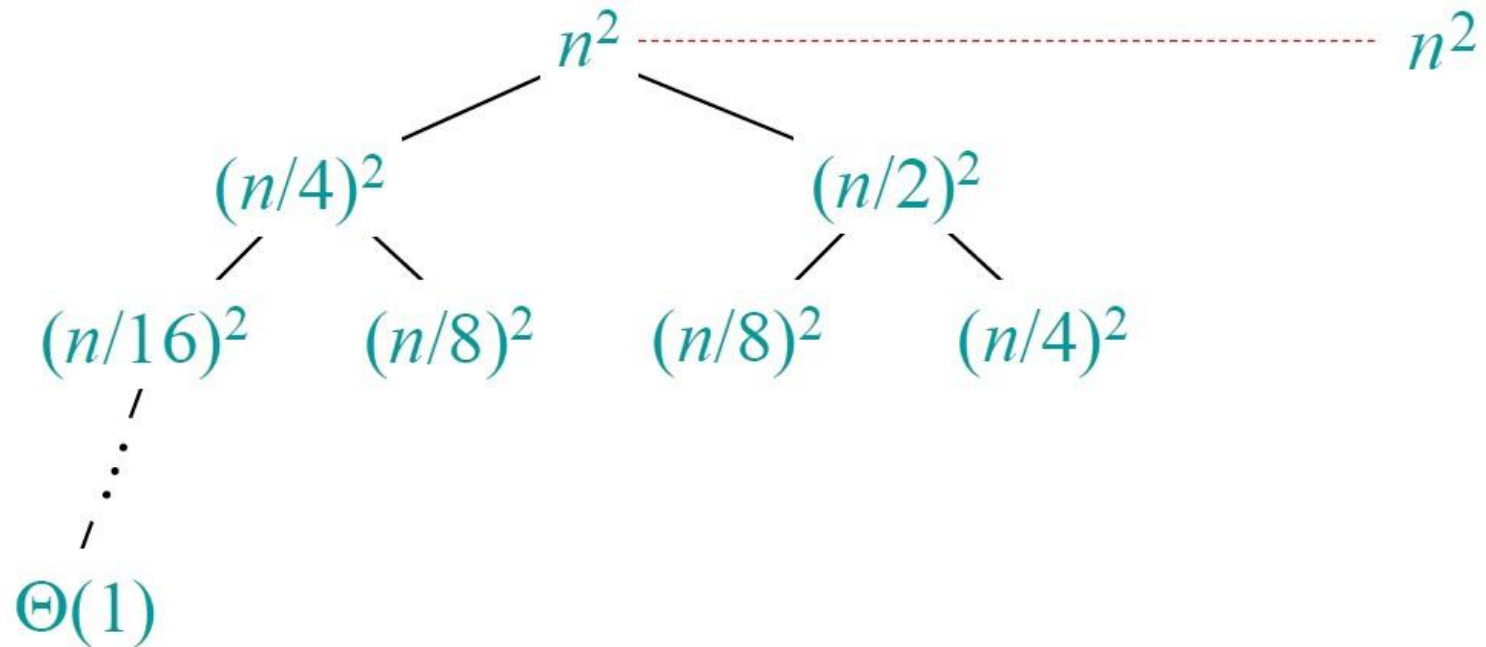
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



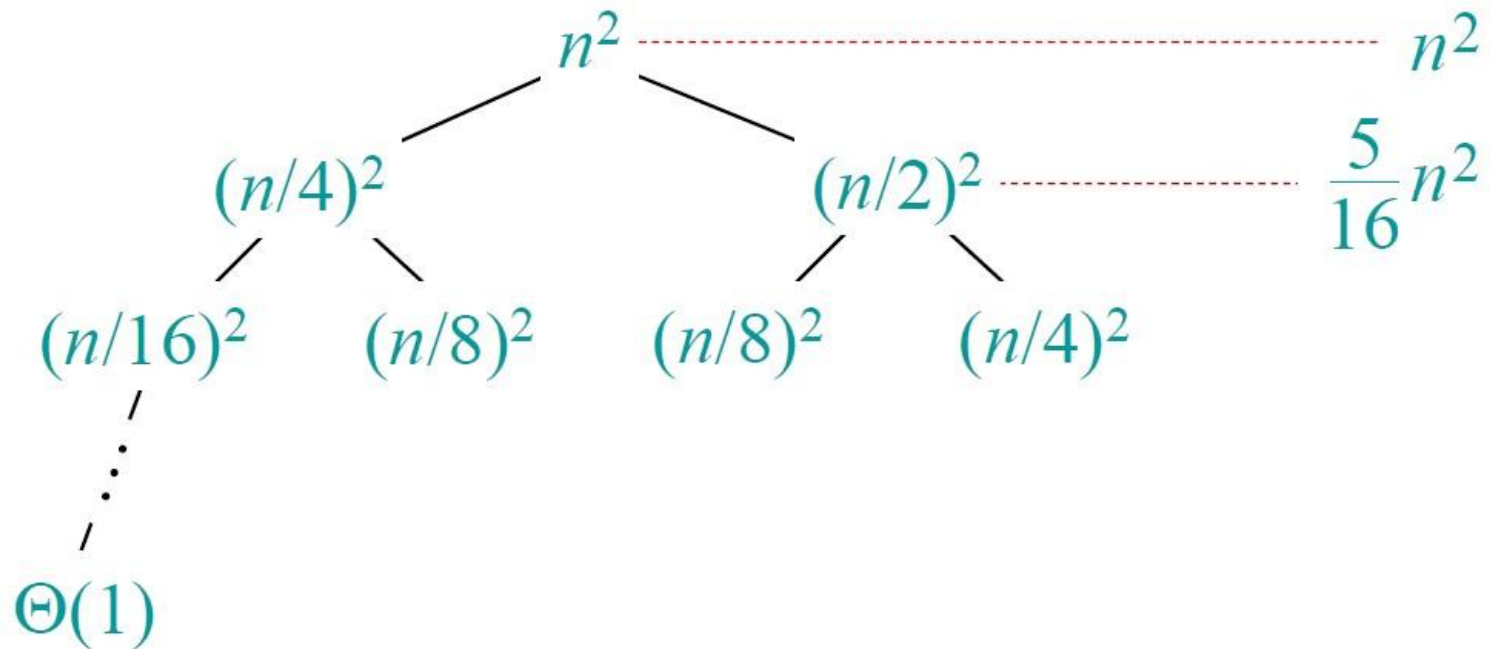
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



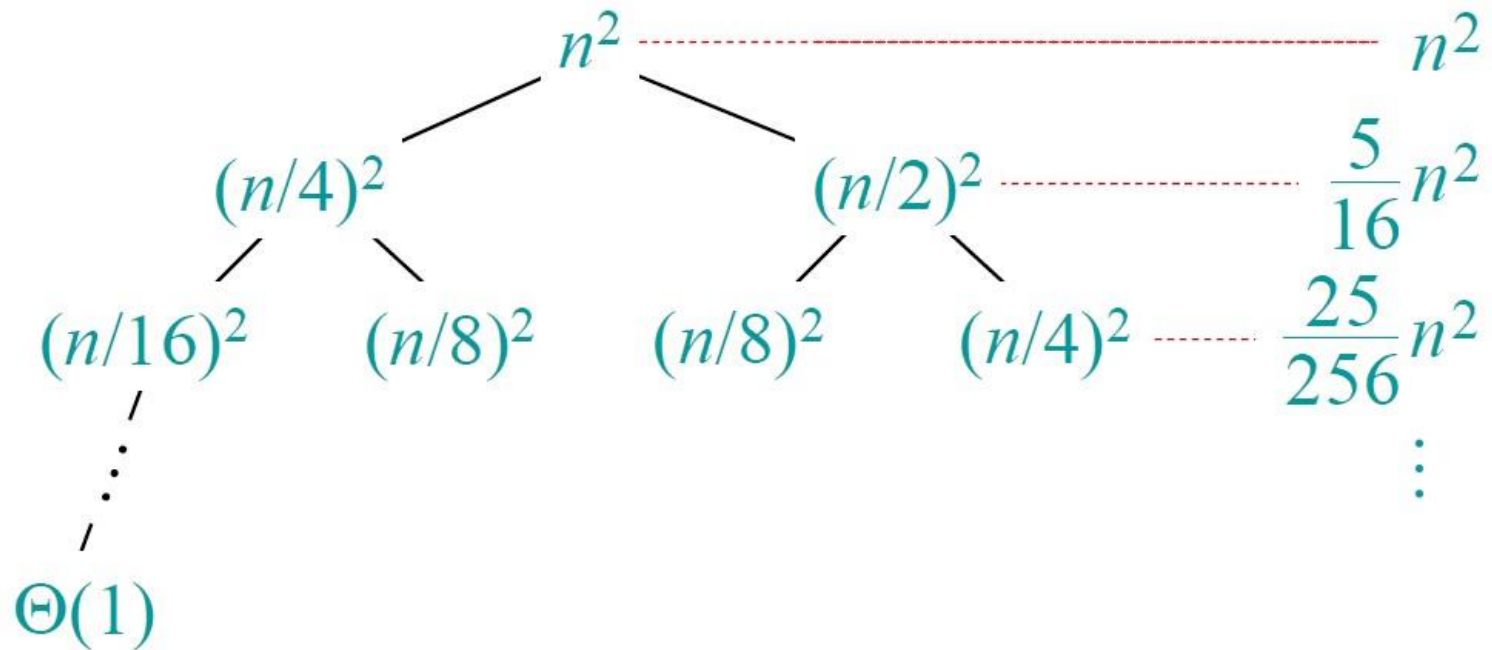
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



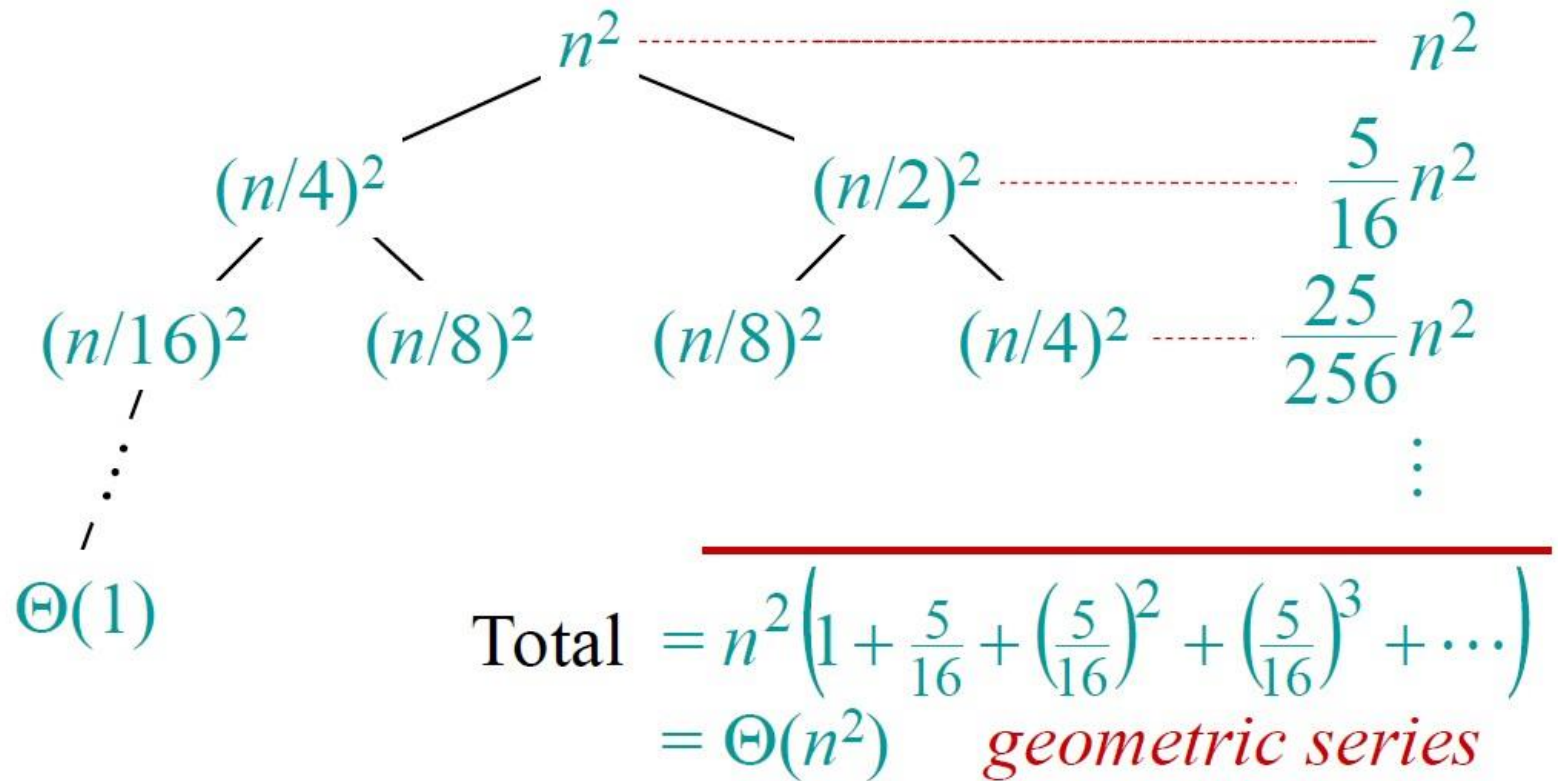
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



Appendix: geometric series

$$1 + x + x^2 + \cdots + x^n = \frac{1 - x^{n+1}}{1 - x} \quad \text{for } x \neq 1$$

$$1 + x + x^2 + \cdots = \frac{1}{1 - x} \quad \text{for } |x| < 1$$

Recursion Tree of $T(n) = T(n/3) + T(2n/3) + O(n)$

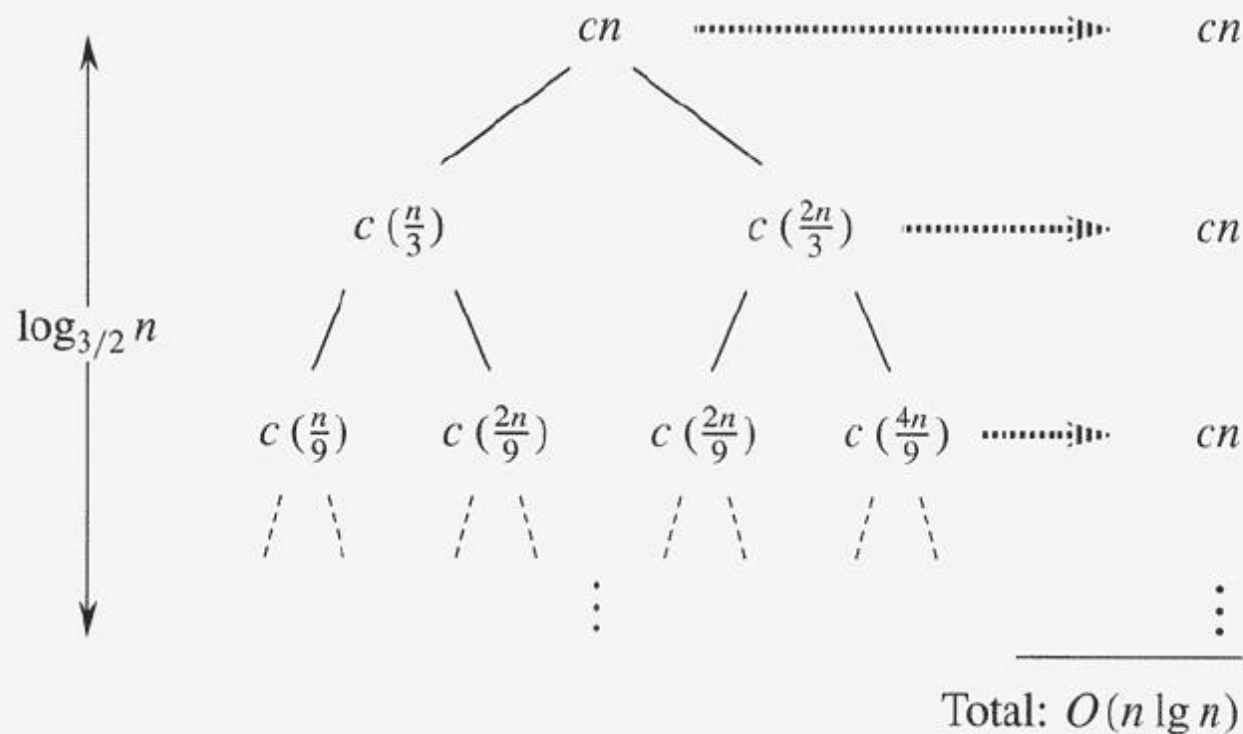


Figure 4.2 A recursion tree for the recurrence $T(n) = T(n/3) + T(2n/3) + cn$.

Master Theorem

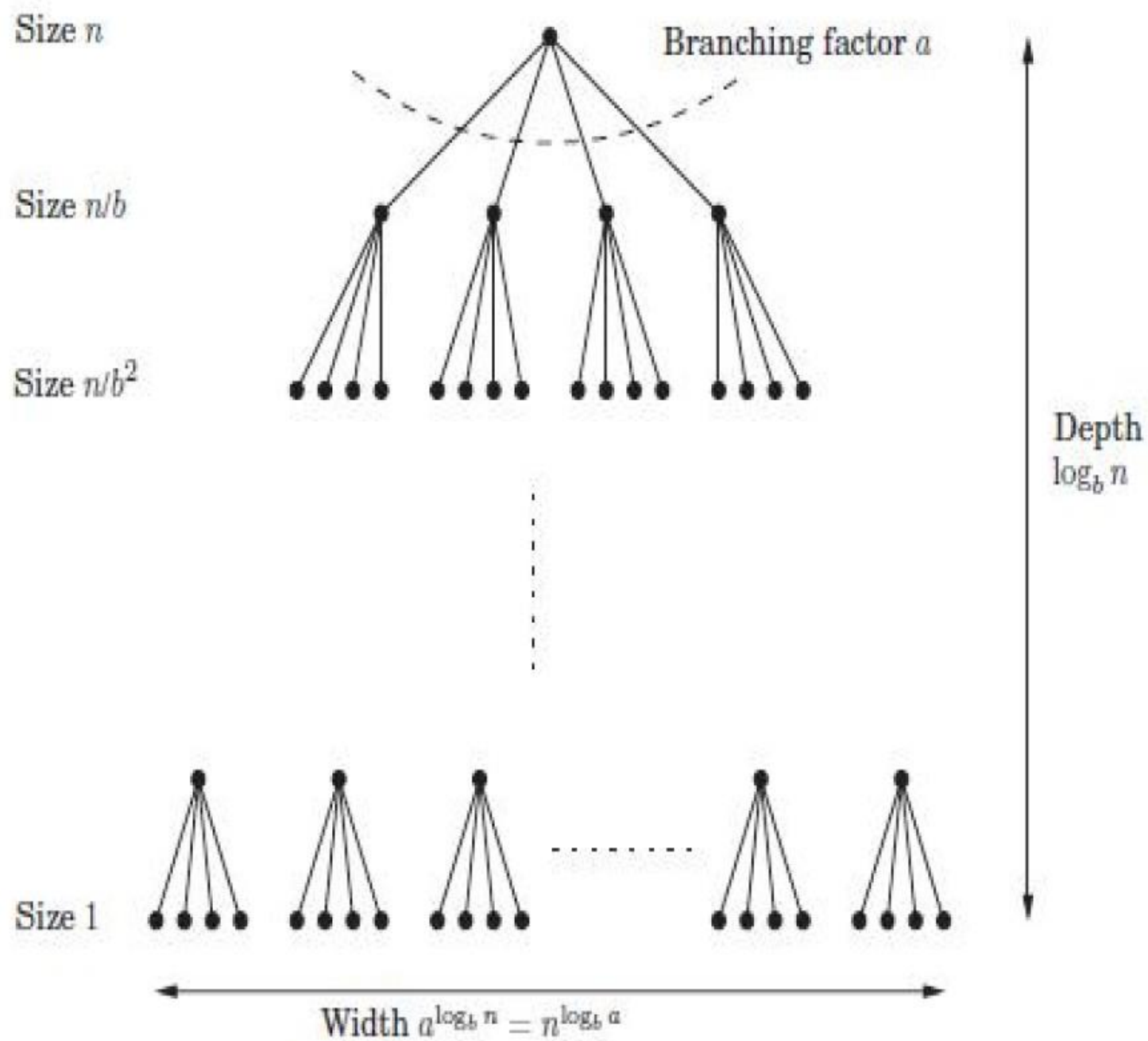
- Let $T(n)$ be a monotonically increasing function that satisfies

$$T(n) = a T(n/b) + f(n)$$

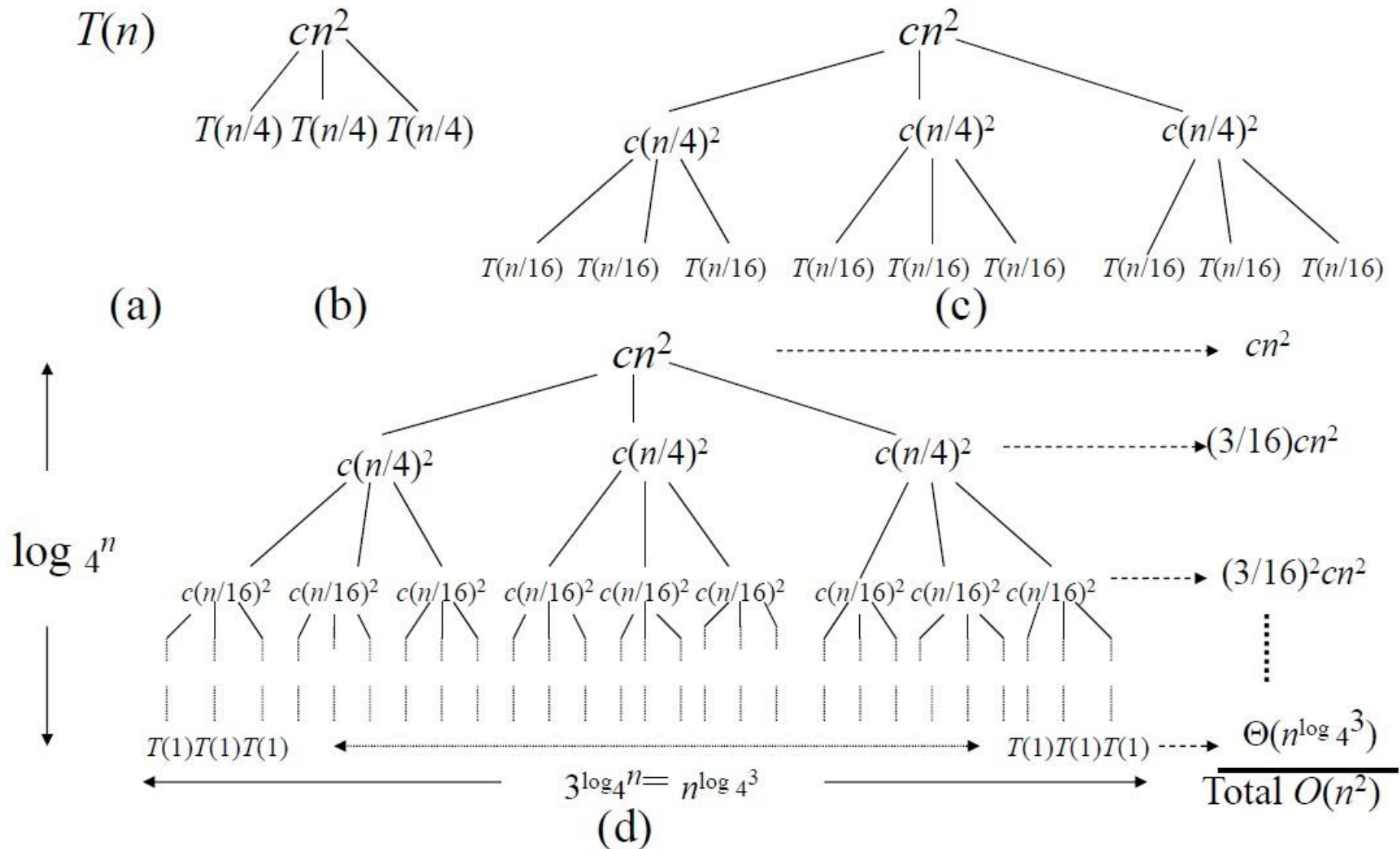
$$T(1) = c$$

where $a \geq 1$, $b \geq 2$, $c > 0$. If $f(n)$ is $\Theta(n^d)$ where $d \geq 0$ then

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } \log_b a < d \text{ or } a < b^d \\ \Theta(n^d \log n) & \text{if } \log_b a = d \text{ or } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } \log_b a > d \text{ or } a > b^d \end{cases}$$



Recursion Tree for $T(n)=3T(\lfloor n/4 \rfloor)+\Theta(n^2)$



Solution to $T(n)=3T(\lfloor n/4 \rfloor)+\Theta(n^2)$

- The height is $\log_4 n$,
- #leaf nodes = $3^{\log_4 n} = n^{\log_4 3}$. Leaf node cost: $T(1)$.
- Total cost $T(n) = cn^2 + (3/16)cn^2 + (3/16)^2 cn^2 + \dots + (3/16)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3})$
 $= (1 + 3/16 + (3/16)^2 + \dots + (3/16)^{\log_4 n - 1}) cn^2 + \Theta(n^{\log_4 3})$
 $< (1 + 3/16 + (3/16)^2 + \dots + (3/16)^m + \dots) cn^2 + \Theta(n^{\log_4 3})$
 $= (1/(1 - 3/16)) cn^2 + \Theta(n^{\log_4 3})$
 $= 16/13 cn^2 + \Theta(n^{\log_4 3})$
 $= O(n^2)$.

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } \log_b a < d \text{ or } a < b^d \\ \Theta(n^d \log n) & \text{if } \log_b a = d \text{ or } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } \log_b a > d \text{ or } a > b^d \end{cases}$$

Solution to $T(n)=3T(\lfloor n/4 \rfloor)+\Theta(n)$

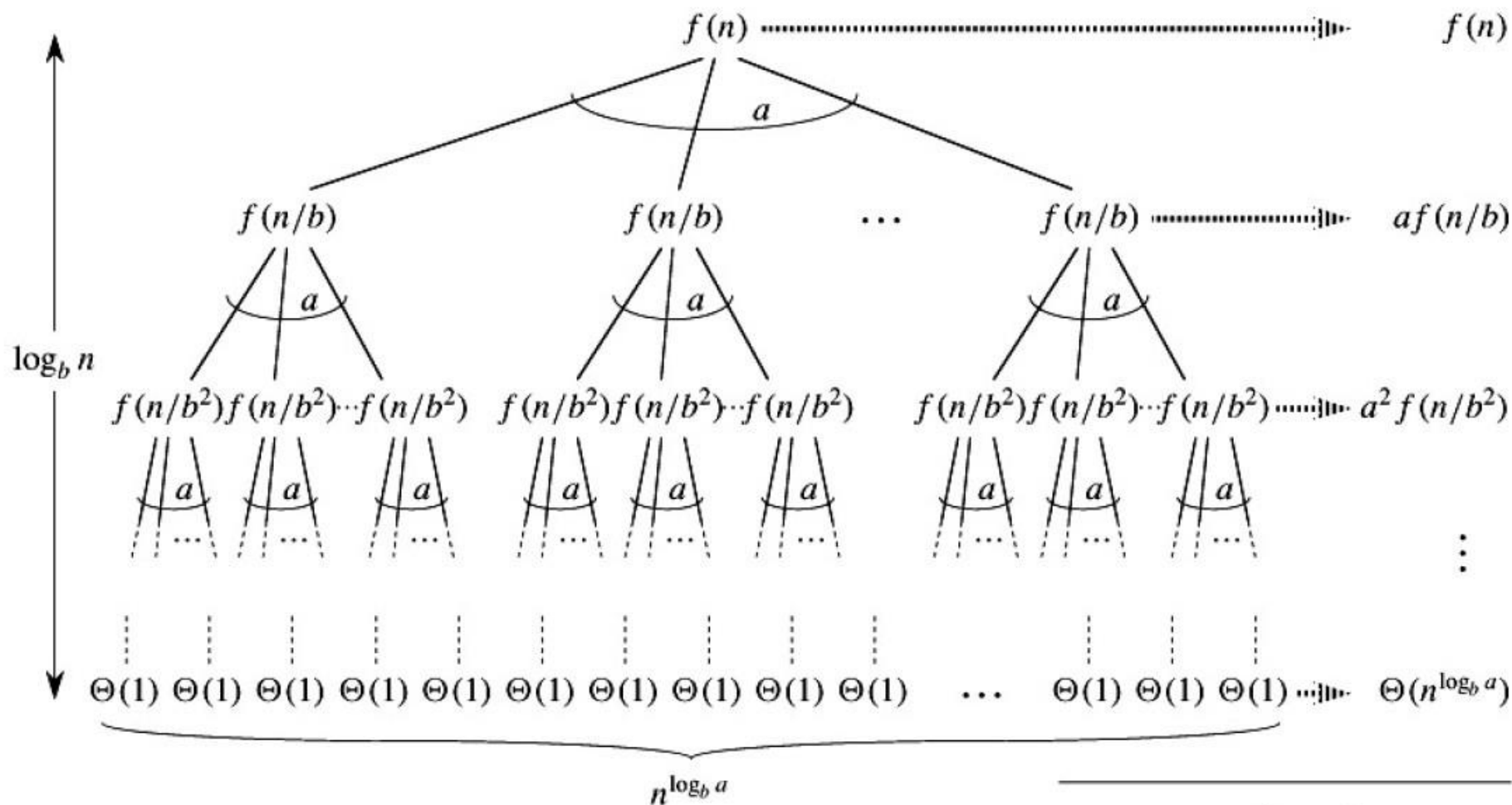
- The height is $\log_4 n$,
- #leaf nodes = $3^{\log_4 n} = n^{\log_4 3}$. Leaf node cost: $T(1)$.
- Total cost $T(n)=cn+(3/16)cn+(3/16)^2cn+\dots+(3/16)^{\log_4 n-1}cn+\Theta(n^{\log_4 3})$
 $= (1+3/16+(3/16)^2+\dots+(3/16)^{\log_4 n-1})cn + \Theta(n^{\log_4 3})$
 $< (1+3/16+(3/16)^2+\dots+(3/16)^m+\dots)cn + \Theta(n^{\log_4 3})$
 $= (1/(1-3/16))cn + \Theta(n^{\log_4 3})$
 $= 16/13cn + \Theta(n^{\log_4 3})$
 $= \underline{\underline{\Theta(n)}}$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } \log_b a < d \text{ or } a < b^d \\ \Theta(n^d \log n) & \text{if } \log_b a = d \text{ or } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } \log_b a > d \text{ or } a > b^d \end{cases}$$

Solution to $T(n)=2T(\lfloor n/4 \rfloor)+\Theta(n)$

- The height is $\log_4 n$,
- #leaf nodes = $2^{\log_4 n} = n^{\log_4 2}$. Leaf node cost: $T(1)$.
- Total cost $T(n)=cn+(2/16)cn+(2/16)^2cn+\dots+(2/16)^{\log_4 n-1}cn+\Theta(n^{\log_4 2})$
 $= (1+2/16+(2/16)^2+\dots+(2/16)^{\log_4 n-1})cn + \Theta(n^{\log_4 2})$
 $< (1+2/16+(2/16)^2+\dots+(2/16)^m+\dots)cn + \Theta(n^{\log_4 2})$
 $= (1/(1-2/16))cn + \Theta(n^{\log_4 2})$
 $= 16/14cn + \Theta(n^{\log_4 2})$
 $= O(n)$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } \log_b a < d \text{ or } a < b^d \\ \Theta(n^d \log n) & \text{if } \log_b a = d \text{ or } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } \log_b a > d \text{ or } a > b^d \end{cases}$$



Basis problems cost $\xrightarrow{\text{red arrow}} \Theta(n^{\log_b a})$

Divide and combine cost $\xrightarrow{\text{green arrow}} \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$

Total: $\Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$

Master Theorem: Example 1

- Let $T(n) = T(n/2) + \frac{1}{2}n^2 + n$. What are the parameters?

$$a = 1$$

$$b = 2$$

$$d = 2$$

Therefore, which condition applies?

$0 < 2$ or $1 < 2^2$, case 1 applies (if $\log_b a < d$ or $a < b^d$)

- We conclude that

$$T(n) \in \Theta(n^d) = \Theta(n^2)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } \log_b a < d \text{ or } a < b^d \\ \Theta(n^d \log n) & \text{if } \log_b a = d \text{ or } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } \log_b a > d \text{ or } a > b^d \end{cases}$$

Master Theorem: Example 2

- Let $T(n) = 2T(n/4) + \sqrt{n} + 42$. What are the parameters?

$$a = 2$$

$$b = 4$$

$$d = 1/2$$

Therefore, which condition applies?

$\frac{1}{2} = \frac{1}{2}$ or $2 = 4^{1/2}$, case 2 applies (if $\log_b a = d$ or $a = b^d$)

- We conclude that

$$T(n) \in \Theta(n^d \log n) = \Theta(\log n \sqrt{n})$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } \log_b a < d \text{ or } a < b^d \\ \Theta(n^d \log n) & \text{if } \log_b a = d \text{ or } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } \log_b a > d \text{ or } a > b^d \end{cases}$$

Master Theorem: Example 3

- Let $T(n) = 3T(n/2) + 3/4n + 1$. What are the parameters?

$$a = 3$$

$$b = 2$$

$$d = 1$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } \log_b a < d \text{ or } a < b^d \\ \Theta(n^d \log n) & \text{if } \log_b a = d \text{ or } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } \log_b a > d \text{ or } a > b^d \end{cases}$$

Therefore, which condition applies?

$\log_2 3 > 1$ or $3 > 2^1$, case 3 applies (if $\log_b a > d$ or $a > b^d$)

- We conclude that

$$T(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2 3})$$

- Note that $\log_2 3 \approx 1.584\dots$, can we say that $T(n) \in \Theta(n^{1.584})$

'Fourth' Condition

- Recall that we cannot use the Master Theorem if $f(n)$, the non-recursive cost, is not a polynomial.
- There is a limited 4th condition of the Master Theorem that allows us to consider polylogarithmic functions.
- **Corollary:** If $f(n) \in \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$ then
$$T(n) \in \Theta(n^{\log_b a} \log^{k+1} n)$$
- This final condition is fairly limited and we present it for sake of completeness.

‘Fourth’ Condition: Example

- Say we have the following recurrence relation

$$T(n) = 2 T(n/2) + n \log n$$

- Clearly, $a=2$, $b=2$, but $f(n)$ is not a polynomial. However, we have $f(n) \in \Theta(n \log n)$, $k=1$
- Therefore, by the 4th condition of the Master Theorem, we can say that

$$T(n) \in \Theta(n^{\log_b a} \log^{k+1} n) = \Theta(n^{\log_2 2} \log^2 n) = \Theta(n \log^2 n)$$

The master method

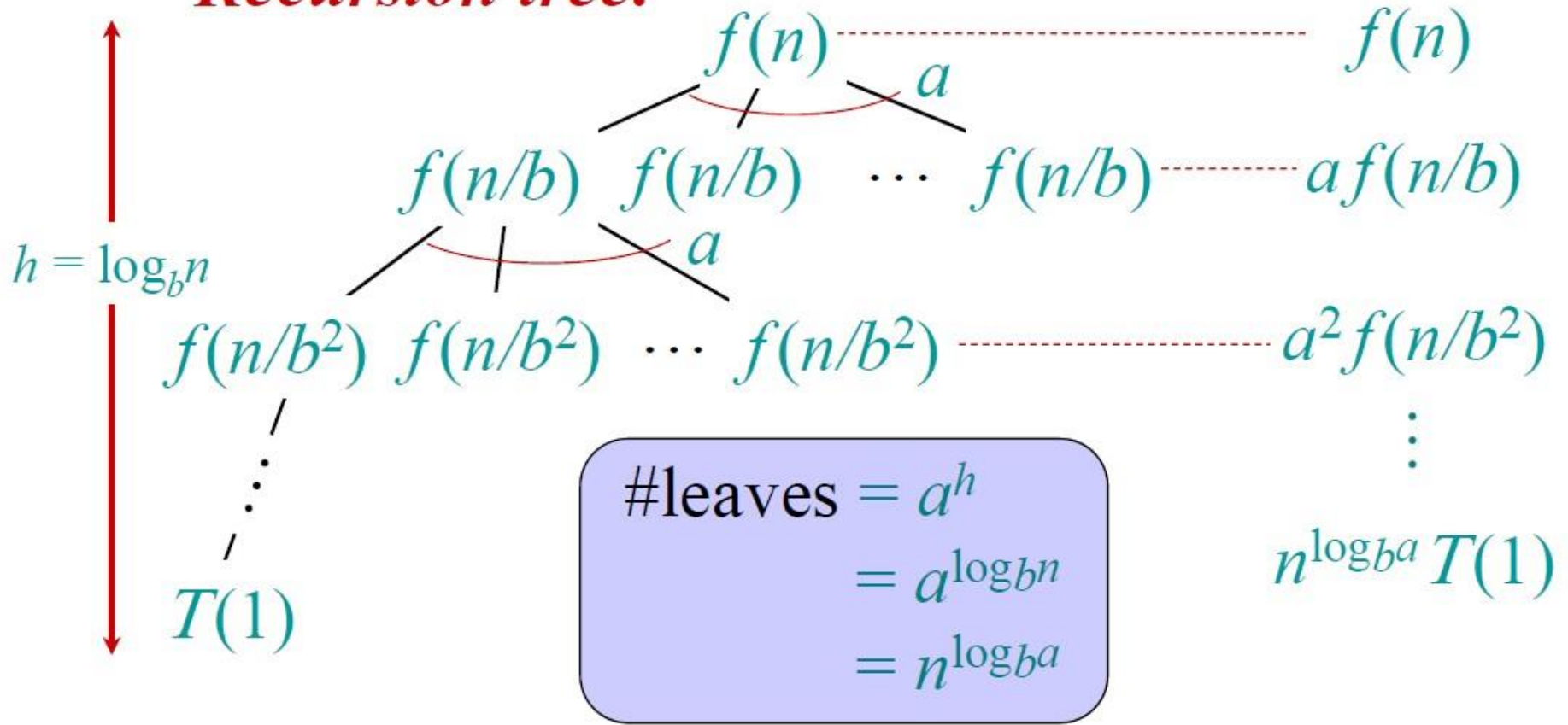
The master method applies to recurrences of the form

$$T(n) = a T(n/b) + f(n) ,$$

where $a \geq 1$, $b > 1$, and f is asymptotically positive.

Idea of master theorem

Recursion tree:



Three common cases

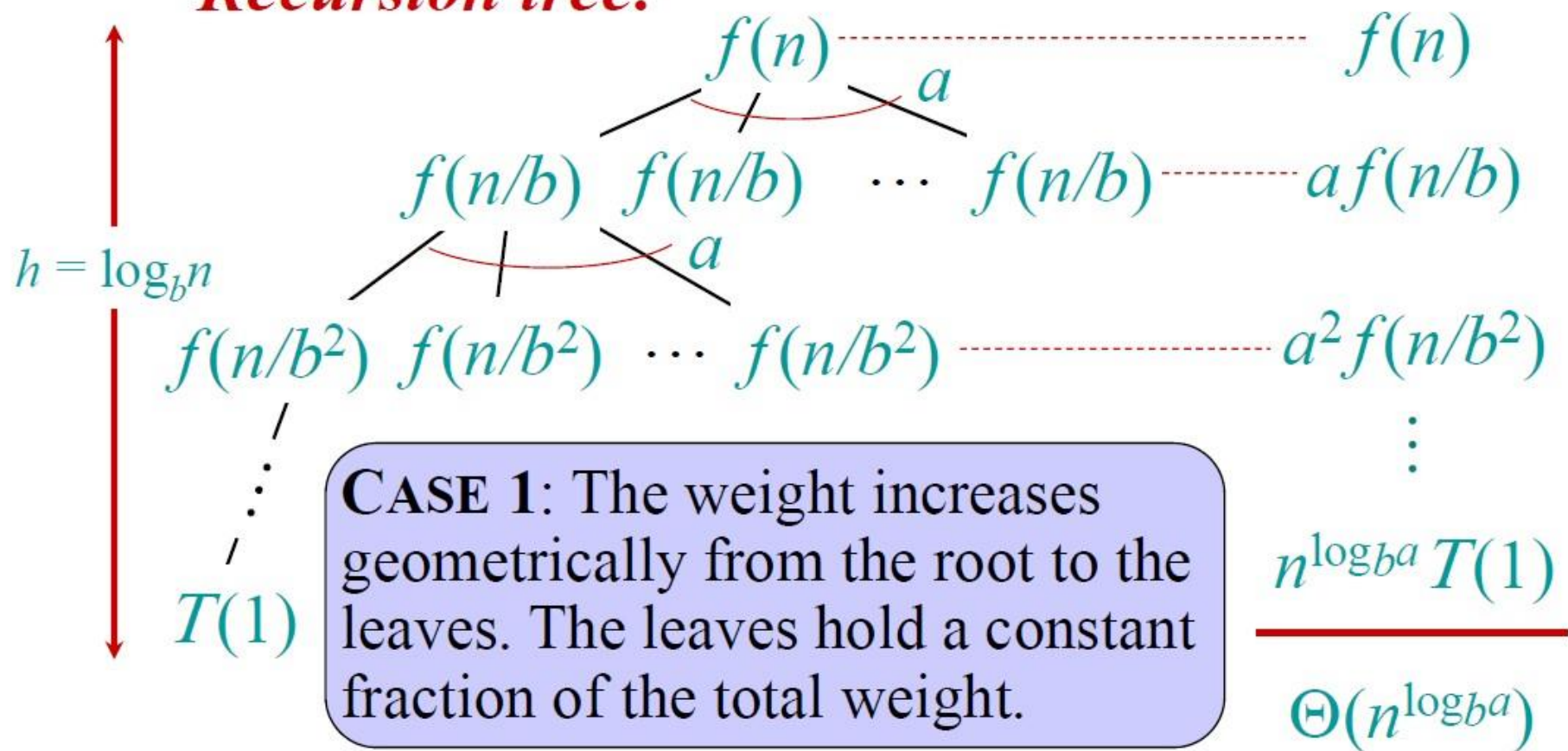
Compare $f(n)$ with $n^{\log_b a}$:

1. $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$.
 - $f(n)$ grows polynomially slower than $n^{\log_b a}$ (by an n^ε factor).

Solution: $T(n) = \Theta(n^{\log_b a})$.

Idea of master theorem

Recursion tree:



Three common cases

Compare $f(n)$ with $n^{\log_b a}$:

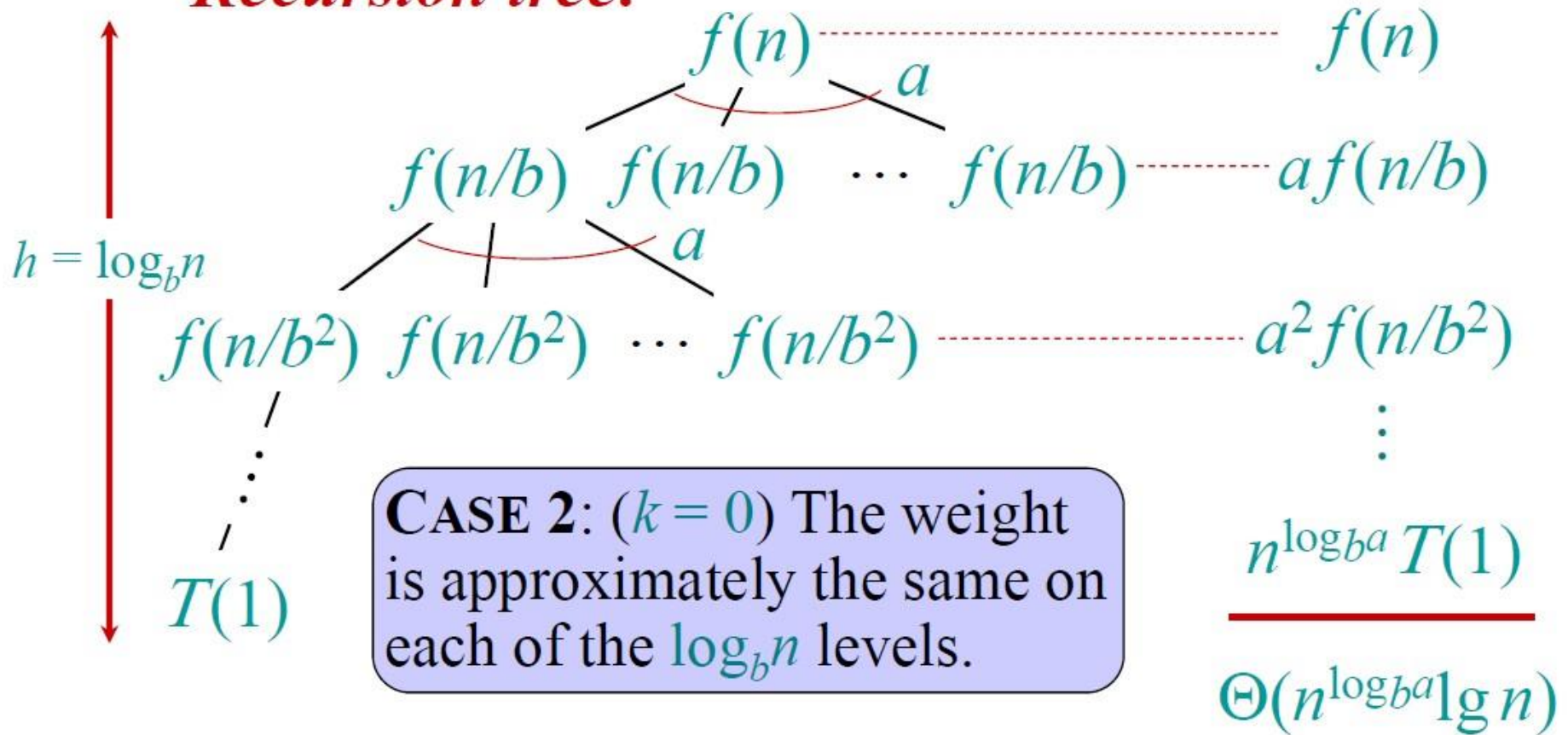
2. $f(n) = \Theta(n^{\log_b a} \lg^k n)$ for some constant $k \geq 0$.

- $f(n)$ and $n^{\log_b a}$ grow at similar rates.

Solution: $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.

Idea of master theorem

Recursion tree:



I Three common cases (cont.)

Compare $f(n)$ with $n^{\log_b a}$:

3. $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$.

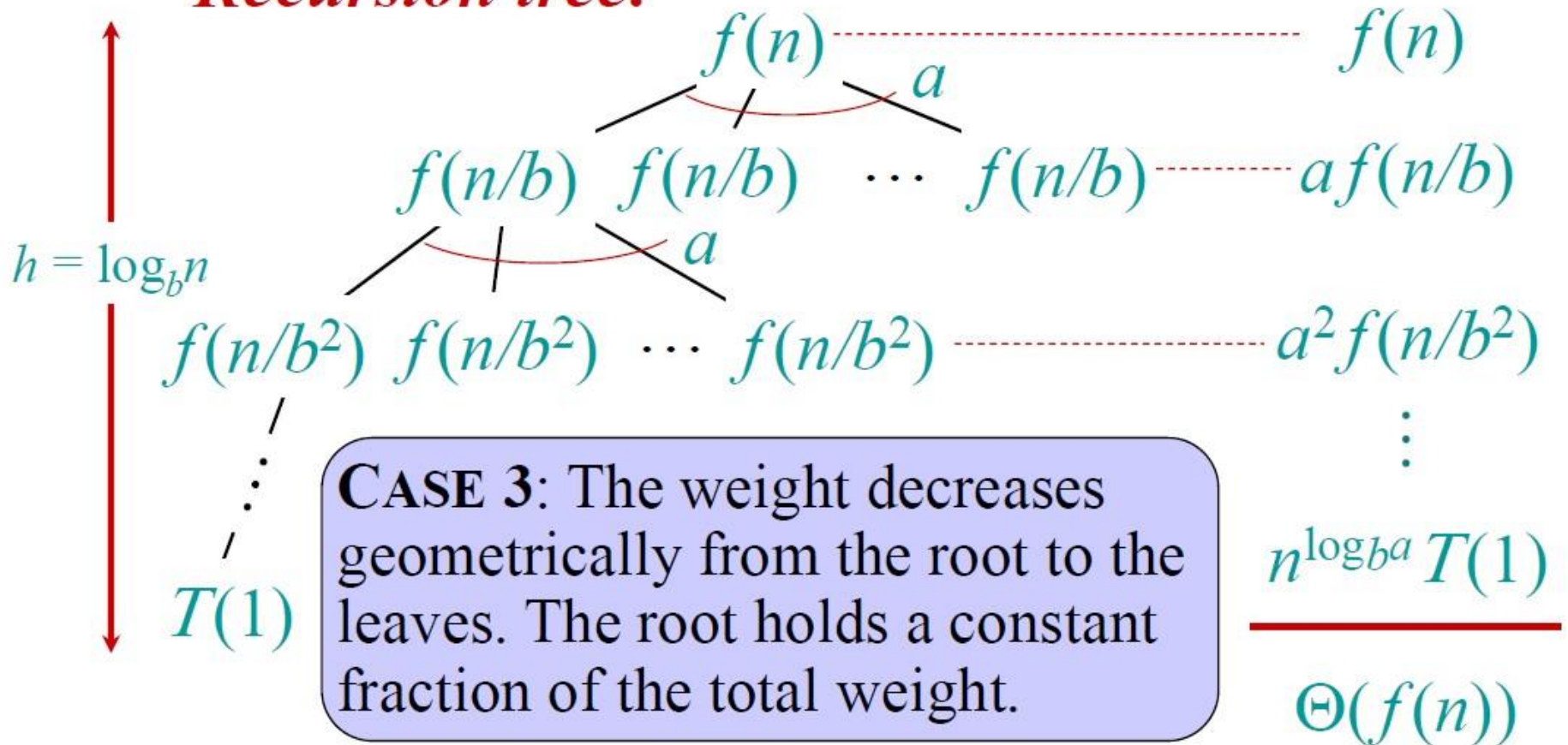
- $f(n)$ grows polynomially faster than $n^{\log_b a}$ (by an n^ε factor),

and $f(n)$ satisfies the **regularity condition** that $a f(n/b) \leq c f(n)$ for some constant $c < 1$.

Solution: $T(n) = \Theta(f(n))$.

Idea of master theorem

Recursion tree:



Examples

I

Ex. $T(n) = 4T(n/2) + n$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n.$$

CASE 1: $f(n) = O(n^{2-\varepsilon})$ for $\varepsilon = 1$.

$$\therefore T(n) = \Theta(n^2).$$

Ex. $T(n) = 4T(n/2) + n^2$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^2.$$

CASE 2: $f(n) = \Theta(n^2 \lg^0 n)$, that is, $k = 0$.

$$\therefore T(n) = \Theta(n^2 \lg n).$$

Examples

[No Title]

Ex. $T(n) = 4T(n/2) + n^3$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^3.$$

CASE 3: $f(n) = \Omega(n^{2+\varepsilon})$ for $\varepsilon = 1$

and $4(cn/2)^3 \leq cn^3$ (reg. cond.) for $c = 1/2$.

$$\therefore T(n) = \Theta(n^3).$$

Ex. $T(n) = 4T(n/2) + n^2/\lg n$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^2/\lg n.$$

Master method does not apply. In particular, for every constant $\varepsilon > 0$, we have $n^\varepsilon = \Omega(\lg n)$.